

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América



FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

“Entregable II”

Curso:

Diseño de Base de Datos

Docente:

Jorge Luis Chávez Soto

Sección: 1

Alumnos:

Bellodas Ramos, Emily Guisell

Quispe Arango, Juan Pablo

Paredes Galvez, Piero Alfonso

Castro Loayza, Yared Benjamin

Vargas Quispe, Sebastian Alexandre

Código:

24200080

24200121

24200026

24200094

24200131

Lima, 11 de Mayo del 2026

CERSEU MEDICINA

1. Perfil del proyecto

1.1. Nombre del proyecto

[Nombre del Proyecto - Por definir]

1.2. Cliente

CERSEU - Facultad de Medicina de la Universidad Nacional Mayor de San Marcos

1.3. Descripción general

El proyecto consiste en el diseño e implementación de una base de datos de nivel empresarial para el Centro de Responsabilidad Social y Extensión Universitaria (CERSEU) de la Facultad de Medicina. El sistema permitirá gestionar de manera integral el proceso de matrícula, control de asistencia y registro de calificaciones de los diversos cursos y programas de extensión.

Un pilar fundamental de la solución es su enfoque en la Responsabilidad Social, alineándose con los Objetivos de Desarrollo Sostenible (ODS), específicamente el ODS 3: Salud y Bienestar, al facilitar el acceso y la gestión de programas educativos que impactan directamente en la salud de la comunidad. Asimismo, la base de datos contará con soporte multimoneda para facilitar pagos nacionales e internacionales.

1.4. Objetivo general

Diseñar e implementar una base de datos relacional profesional que automatice y optimice los procesos académicos y administrativos del CERSEU de la Facultad de Medicina, garantizando la integridad, seguridad y escalabilidad de la información bajo estándares corporativos.

1.5. Objetivos específicos

- Centralizar el registro de estudiantes, docentes y la oferta académica de los cursos de extensión de Medicina.
- Automatizar el seguimiento de la asistencia y la carga de notas para mejorar la eficiencia operativa del centro.
- Implementar una arquitectura de datos que soporte múltiples divisas (Soles, Dólares, Euros) para el registro de matrículas.
- Asegurar que la estructura de datos permita la trazabilidad de los proyectos de

responsabilidad social vinculados a las metas de salud pública de la ONU.

- Desarrollar la lógica de negocio íntegramente en el idioma inglés para cumplir con los estándares técnicos globales solicitados.

1.6. Alcance del proyecto

El proyecto abarca el ciclo de vida de la base de datos, incluyendo:

- Modelado de procesos de negocio (BPMN) y definición de reglas de negocio.
- Diseño del modelo conceptual de datos.
- Implementación de scripts en SQL (MySQL para entregas parciales y Oracle para la final) para la creación de tablas, restricciones y objetos de programación (triggers, procedimientos, funciones).
- Carga de datos ficticios para pruebas de funcionalidad y estrés.
- Definición de políticas de respaldo y seguridad de la información.

1.7. Limitaciones

- El desarrollo se centra exclusivamente en el Backend (Base de Datos y lógica de servidor en SQL).
- No incluye el desarrollo de una interfaz gráfica de usuario (Frontend) ni aplicaciones móviles.
- No se contempla la integración con sistemas externos de la universidad (como el SUM) en esta primera fase.

2. Gestión del proyecto

2.1. Plan de trabajo

El equipo ha establecido un cronograma de actividades orientado al cumplimiento de los hitos del curso, simulando un entorno de consultoría profesional bajo presión y plazos estrictos. A continuación, se detalla la distribución de tareas y tiempos estimados:

Item	Actividad	Responsable	Tiempo estimado
1	Definición del cliente y análisis del caso CERSEU Medicina	Todo el equipo	1 día
2	Elaboración del perfil del proyecto	Sebastian Vargas	2 días

3	Elaboración del Plan de trabajo y cronograma	Piero Paredes	1 día
4	Descripción de la organización: CERSEU Medicina	Juan Pablo Quispe	2 días
5	Identificación de procesos críticos de negocio	Emily Bellodas	1 día
6	Elaboración de diagramas BPMN generales	Yared Castro	1 día
7	Documentación BPMN del proceso de Matrícula	Piero Paredes	2 horas
8	Documentación BPMN del proceso de Asistencia	Sebastian Vargas	2 horas
9	Documentación BPMN del proceso de Registro de Notas	Emily Bellodas	2 horas
10	Formulación de reglas de negocio (multimoneda y ODS)	Juan Pablo Quispe	5 horas
11	Diseño del modelo conceptual de datos (E-R)	Yared Castro	2 semanas
12	Revisión y ajustes del modelo conceptual	Sebastian Vargas	2 días
13	Diseño del modelo lógico (relacional)	Juan Pablo Quispe	3 días
14	Diseño del modelo físico y scripts de creación (SQL)	Sebastian Vargas	3 días
15	Carga de datos ficticios y validación inicial	Emily Bellodas	2 días
16	Elaboración de scripts de prueba de funcionalidades	Piero Paredes	1 día.
17	Preparación de informes Técnico y Ejecutivo	Yared Castro	2 semanas
18	Compilación y empaquetado del proyecto final	Todo el equipo	1 día

2.2. Roles y responsabilidades del equipo

Para garantizar una ejecución eficiente, se han definido roles específicos basados en

las competencias de cada miembro, asegurando una división de trabajo balanceada y colaborativa:

Item	Integrante	Rol	Tareas
1	Piero Paredes	Coordinador	Supervisión general, comunicación con el docente, y aseguramiento de plazos.
2	Emily Bellodas	Analista de requisitos	Recopilación de necesidades del CERSEU, definición de procesos de negocio y validación de reglas de negocio.
3	Sebastian Vargas	Diseñador lógico y de scripts	Transformación del modelo a nivel físico, desarrollo de scripts en SQL y configuración de restricciones en la base de datos.
4	Yared Castro	Diseñador conceptual / Documentador	Diseño del Modelo Entidad-Relación, redacción de los informes corporativos y consolidación final del estilo del documento.
5	Juan Pablo Quispe	Diseñador del modelo lógico	Traducción del modelo conceptual al relacional, normalización de tablas y definición de llaves primarias y foráneas.

2.3. Herramientas y seguimiento

Para garantizar el cumplimiento de los plazos, la correcta división de las tareas y la calidad de los scripts de la base de datos para el CERSEU Medicina, el equipo utilizará un conjunto de herramientas tecnológicas especializadas e aplicará una metodología de seguimiento rigurosa.

A. Herramientas Tecnológicas

Para mantener la trazabilidad y eficiencia en las etapas de diseño y desarrollo, se ha definido el siguiente stack de herramientas:

Categoría	Herramienta	Propósito en el Proyecto
Modelado de Procesos	PlanUML	Creación estandarizada de los diagramas BPMN para documentar los procesos críticos de matrícula, asistencia y registro de calificaciones.
Diseño de Datos	Erwin Data Modeler / Lucidchart	Estructuración visual de los modelos conceptuales (Entidad-Relación) y lógicos, garantizando una correcta normalización y definición de llaves.
Gestores de Base de Datos (SGBD)	MySQL Workbench / Oracle SQL Developer	IDEs oficiales para la escritura, prueba y ejecución de los scripts de creación de esquemas, procedimientos, funciones y triggers (MySQL para las etapas parciales y Oracle para el entregable final).
Administración Multiplataforma	DBeaver	Cliente SQL universal que servirá como herramienta de apoyo para la gestión ágil de las bases de datos, ideal por su alta eficiencia técnica y compatibilidad nativa con entornos de desarrollo basados tanto en distribuciones Linux como en Windows.
Control de Versiones	Git y GitHub	Repositorio centralizado para almacenar y versionar el código fuente. Asegurará que los scripts de carga de datos y creación de objetos no se sobrescriban entre los desarrolladores.
Gestión de Tareas	Trello (Tablero Kanban)	Gestión visual de las actividades del

		cronograma, permitiendo mover tarjetas de tareas entre estados: Pendiente, En Desarrollo, Pruebas y Finalizado.
Comunicación y Soporte	Discord / WhatsApp	Canales oficiales para la comunicación del equipo. Se emplearán canales de voz y compartición de pantalla en tiempo real para depurar scripts conjuntamente y resolver conflictos de lógica de servidor.

B. Metodología de Seguimiento

El control del proyecto se ejecutará simulando un entorno de consultoría bajo los siguientes protocolos operativos:

- Sincronización Continua: El Coordinador (Piero Paredes) realizará reuniones breves de control dos veces por semana para verificar el avance de cada responsable según el cronograma establecido en el ítem 2.1, identificando bloqueos técnicos de forma temprana.
- Control de Cambios Estricto: Todo script SQL generado (DDL o DML) deberá cumplir con la regla de negocio RN-42 (nomenclatura obligatoria en idioma inglés). Las modificaciones en el código deberán subirse al repositorio de control de versiones con un mensaje descriptivo claro.
- Auditoría de Integración: Antes de la compilación y empaquetado del proyecto final (ítem 18 del cronograma), el equipo realizará pruebas cruzadas. Los diseñadores del modelo lógico y físico contrastarán los scripts finales contra las reglas de negocio (específicamente la lógica multimonedas y el cálculo de ponderados) para asegurar que no existan discrepancias entre el diseño conceptual y la implementación en Oracle.

3. La empresa

3.1. Empresa cliente

La empresa cliente es el Centro de Responsabilidad Social y Extensión Universitaria (CERSEU) de la Facultad de Medicina de la Universidad Nacional Mayor de San Marcos. Esta unidad orgánica es el puente principal entre la academia y la sociedad, encargada de promover la extensión universitaria y la proyección social para fortalecer el vínculo institucional con la comunidad.

El CERSEU Medicina requiere una solución tecnológica robusta para gestionar su creciente oferta académica, que incluye programas de capacitación, talleres de primeros auxilios (RCP y OVACE), webinars de salud mental y proyectos de voluntariado. El sistema debe permitir la transición de procesos manuales a una base de datos automatizada que soporte el registro de matrículas en diversas monedas y el seguimiento detallado de la asistencia y notas de los participantes.



UNMSM
FACULTAD DE MEDICINA
SAN FERNANDO

3.2. Descripción de la organización

Direcciones y Áreas de la Organización

La estructura del CERSEU Medicina se organiza en función de sus programas y servicios estratégicos, los cuales se dividen en las siguientes áreas funcionales:

- **Dirección:** Encargada de la planificación estratégica, representación institucional y toma de decisiones de alto nivel.
- **Gestión académica de extensión:** Área responsable de la programación de cursos, foros, seminarios y talleres dirigidos a la comunidad universitaria y público en general.
- **Proyección social y voluntariado:** Encargada de las acciones de fortalecimiento del voluntariado y proyectos de Responsabilidad Social

Universitaria.

- **Aprendizaje servicio (APS):** Unidad que articula las prácticas académicas con el servicio directo a comunidades vulnerables.
- **Gestión administrativa:** Responsable del soporte logístico, secretaría, atención al estudiante y procesos de matrícula y pagos.

Personal de la organización

Cargo/Área	Cantidad	Funciones principales
Directora del CERSEU	1	Liderazgo institucional, supervisión de proyectos de investigación y extensión
Comité de Directivos	1	Representantes de las Escuelas Profesionales (Medicina Humana, Nutrición, Enfermería, Obstetricia y Tecnología Médica) que validan la calidad académica.
Secretaría	9	Gestión documentaria, atención al público y soporte administrativo directo.
Apoyo Administrativo	1	Ejecución de procesos operativos, registro de participantes y apoyo logístico.

Volumen de Operaciones (Estimado)

De acuerdo con los reportes de gestión recientes (Semestre 2025-II), la organización maneja los siguientes volúmenes de datos:

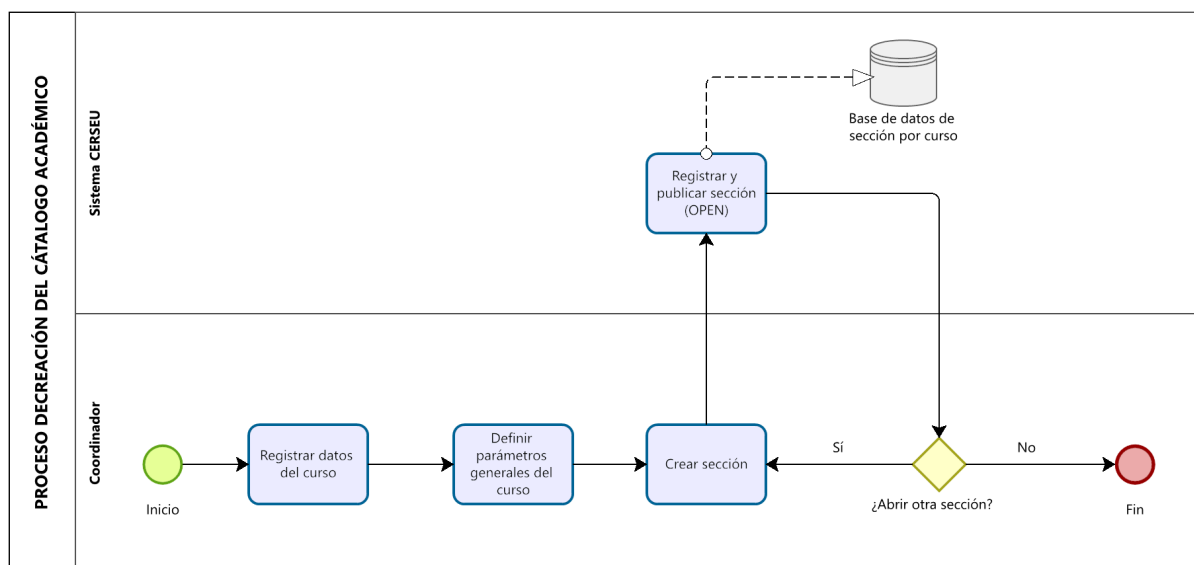
- **Programas:** Aproximadamente 19 programas de extensión por semestre académico.
- **Participantes:** Un promedio de 438 alumnos matriculados por ciclo en diversas modalidades (presencial, virtual e híbrida).
- **Alcance:** Cobertura nacional e internacional, lo que justifica la implementación de una arquitectura multimonedada para los registros financieros.

4. Modelo de Procesos de Negocio

A continuación, se presentan los diagramas BPMN que describen los procesos clave del sistema CERSEU Medicina, detallando el flujo desde la matrícula hasta el registro de calificaciones. Estos permiten visualizar las interacciones entre los actores, las tareas automatizadas y los resultados generados en cada etapa.

4.1. Creación del Catálogo Académico

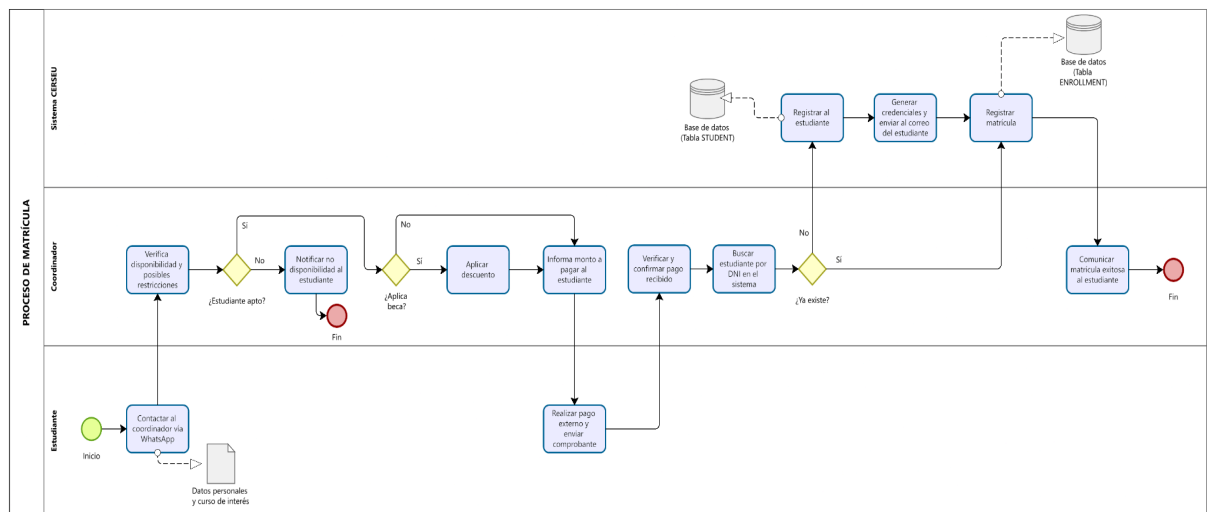
Este proceso describe cómo el coordinador del CERSEU registra la oferta académica en el sistema. Inicia con la creación de un curso de extensión, donde se definen sus datos generales (nombre, descripción, total de horas y área de salud) y se carga el sílabo con los temas organizados por sesión. Posteriormente se configuran los parámetros generales del curso (mínimo y máximo de estudiantes, plataforma virtual) y se crea al menos una sección, asignándole un docente, fechas de inicio y fin, duración y horario. Una vez completada la configuración, el sistema registra la sección en la base de datos y la publica con estado OPEN, quedando disponible para recibir matrículas. El coordinador puede abrir tantas secciones como la demanda lo requiera.



4.2. Admisión y Matrícula

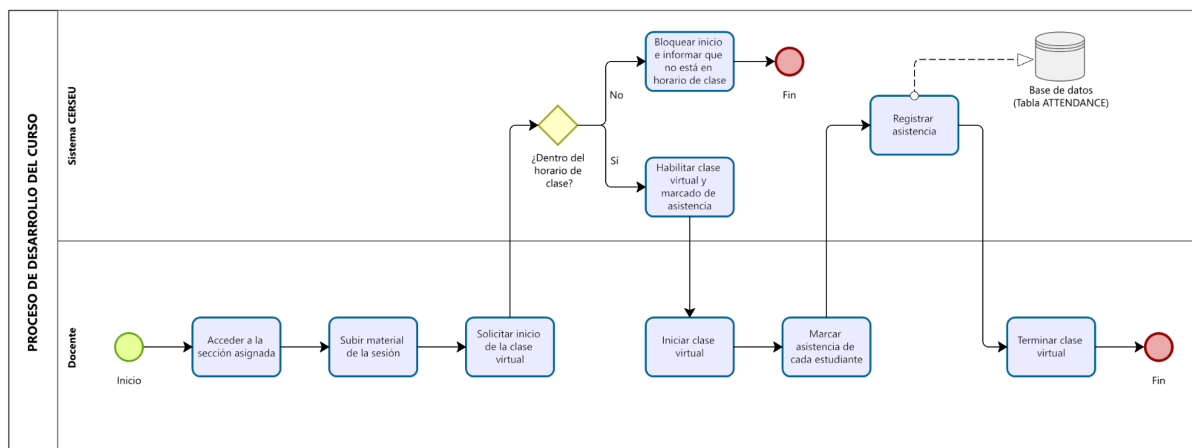
Este proceso gestiona el ingreso de un estudiante a un curso de extensión. Comienza

cuando el estudiante contacta al coordinador vía WhatsApp manifestando su interés. El coordinador verifica en el sistema la disponibilidad y posibles restricciones (cupo, choque de horarios, duplicidad) antes de proceder. Si el estudiante es apto, se le informa el monto a pagar, aplicando una beca si corresponde. Tras confirmar el pago externo, el coordinador busca al estudiante en el sistema: si es nuevo, registra sus datos personales y el sistema genera y envía sus credenciales de acceso por correo; si ya existe, solo lo selecciona. Finalmente, el sistema registra la matrícula en la base de datos, formalizando la inscripción.



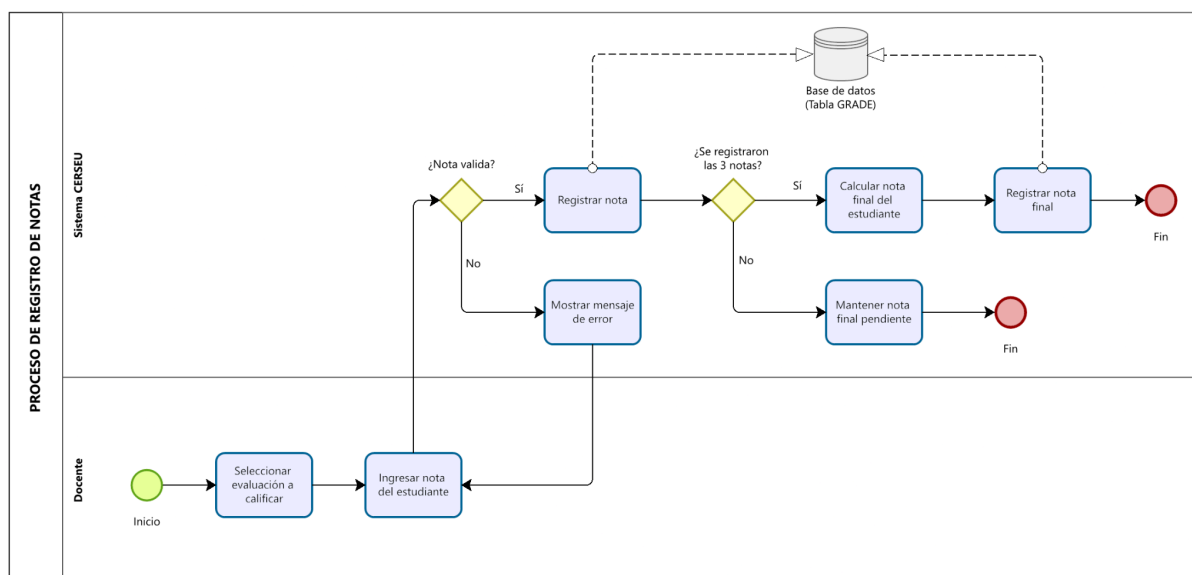
4.3. Gestión de Sesión de Clase

Este proceso describe las actividades del docente durante cada sesión del curso. El docente accede a su sección asignada y sube el material correspondiente a la sesión (diapositivas, PDF, video o enlaces). Al solicitar el inicio de la clase virtual, el sistema valida que se encuentre dentro del horario establecido; si no lo está, bloquea el acceso. Si el horario es correcto, el sistema habilita la clase y el marcado de asistencia, permitiendo al docente registrar la asistencia de cada estudiante (presente, ausente, tardanza o justificado). El sistema almacena esta información en la base de datos. Este proceso se repite en cada sesión programada del curso.



4.4. Registro de Calificaciones

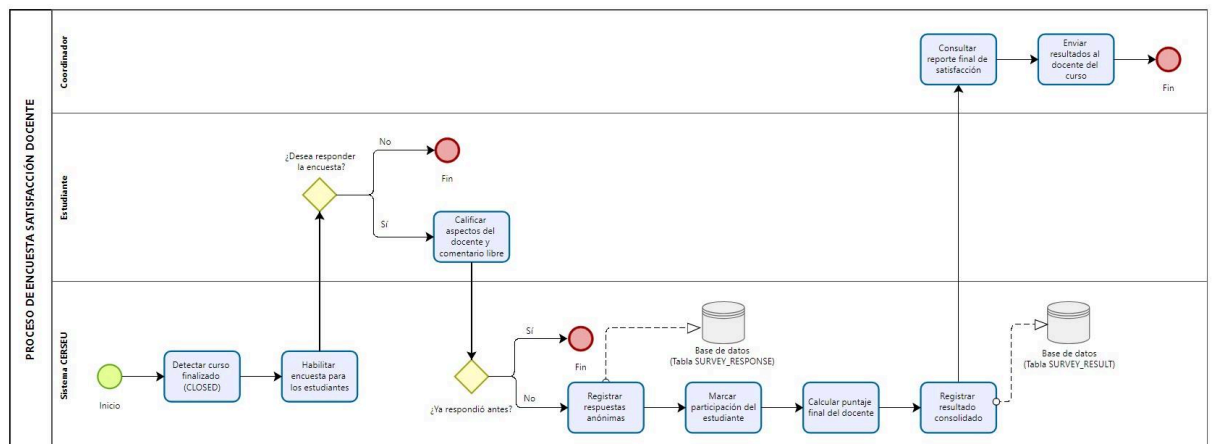
Este proceso detalla cómo el docente registra las notas de los estudiantes a lo largo del curso. El docente selecciona la evaluación a calificar (Parcial 30%, Continua 40% o Final 30%) e ingresa la nota del estudiante. El sistema valida que la nota se encuentre en el rango permitido de 0 a 20; si es inválida, muestra un mensaje de error y solicita reingresarla. Una vez registradas las tres notas, el sistema calcula automáticamente la nota final ponderada y la almacena en la base de datos. Mientras falten notas, la nota final permanece pendiente.



4.5. Encuesta de Satisfacción Docente

Este proceso permite recoger la opinión de los estudiantes sobre el desempeño del docente al finalizar el curso. El sistema habilita una encuesta opcional para los

estudiantes matriculados, sin importar si aprobaron o no. El estudiante califica varios aspectos del docente en una escala del 1 al 5 y puede dejar un comentario libre. El sistema verifica que el estudiante no haya respondido previamente (la encuesta se responde una sola vez), registra las respuestas de forma anónima y calcula el puntaje final del docente como promedio de los aspectos evaluados. Finalmente, el coordinador consulta el reporte consolidado y envía los resultados al docente correspondiente.



5. Reglas del negocio

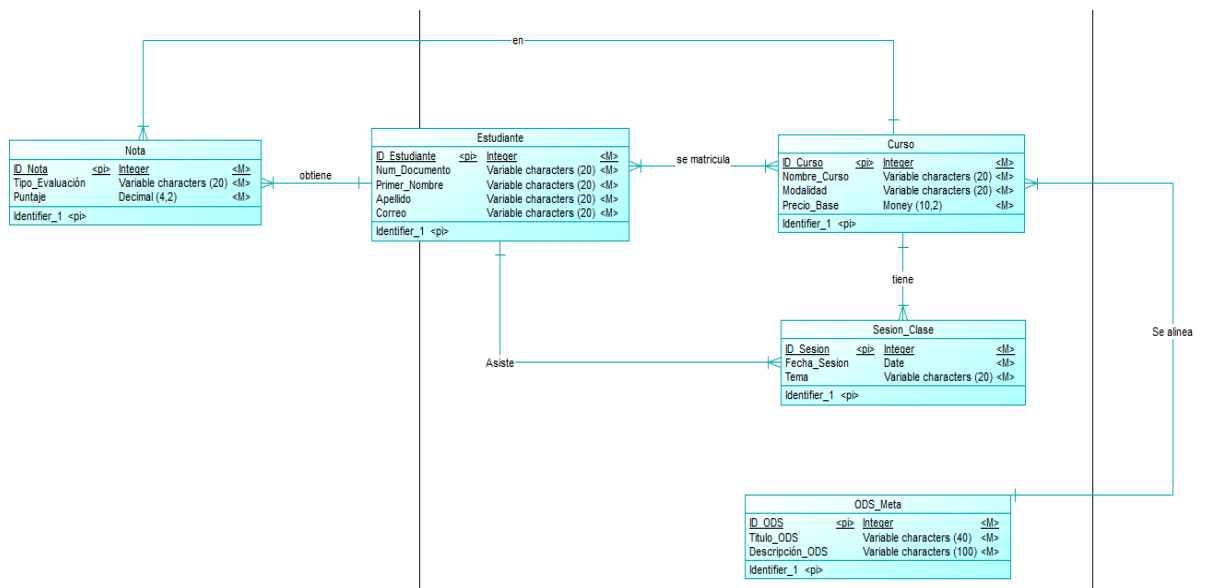
1. **RN-01.** El código de cada curso es único.
2. **RN-02.** Un curso puede tener una o más secciones según la demanda de estudiantes.
3. **RN-03.** Cada sección tiene asignado un único docente.
4. **RN-04.** Un mismo docente puede estar asignado a varias secciones.
5. **RN-05.** El mínimo y máximo de estudiantes, así como la plataforma virtual, son parámetros generales del curso e iguales para todas sus secciones.
6. **RN-06.** La fecha de fin de una sección debe ser posterior a su fecha de inicio.
7. **RN-07.** Una sección solo puede recibir matrículas cuando su estado es OPEN.

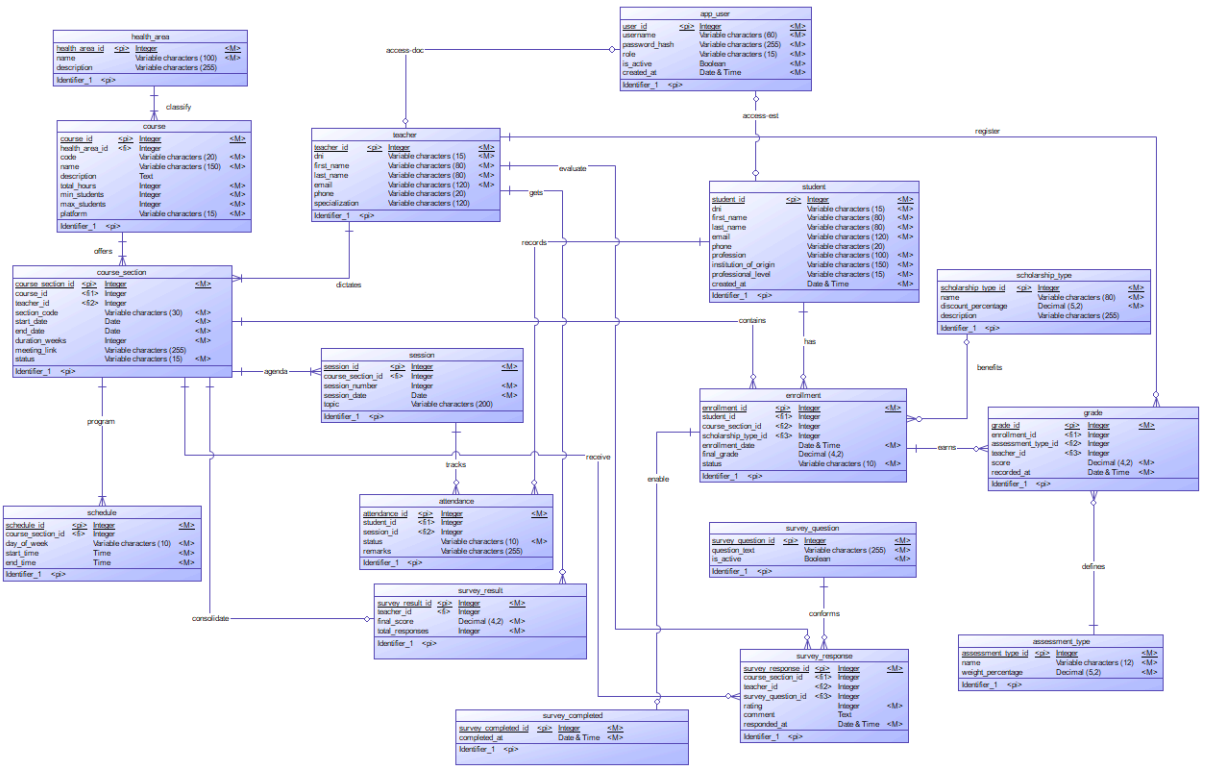
8. **RN-08.** Los estados posibles de una sección son: OPEN, IN_PROGRESS, CLOSED y CANCELLED.
9. **RN-09.** Una sección que no alcanza el mínimo de estudiantes puede ser cancelada (estado CANCELLED).
10. **RN-10.** Los cursos del CERSEU son 100% virtuales.
11. **RN-11.** El DNI de cada estudiante es único en el sistema.
12. **RN-12.** El correo electrónico de cada estudiante es único y obligatorio (es el medio de envío de credenciales y certificado).
13. **RN-13.** Los datos personales obligatorios del estudiante son: nombre, DNI, correo, profesión, institución de origen y nivel profesional.
14. **RN-14.** La disponibilidad y las restricciones se verifican antes de que el estudiante realice el pago, para evitar devoluciones.
15. **RN-15.** Un estudiante no puede matricularse en una sección cuyo horario se cruce con otra de sus matrículas activas.
16. **RN-16.** Un estudiante no puede matricularse dos veces en la misma sección.
17. **RN-17.** No se puede matricular a un estudiante si la sección alcanzó su cupo máximo.
18. **RN-18.** El pago es externo y no lo gestiona el sistema; el sistema solo registra la matrícula una vez confirmado el pago.
19. **RN-19.** Un estudiante nuevo recibe credenciales de acceso automáticamente por correo; un estudiante existente conserva sus credenciales previas.
20. **RN-20.** Una beca aplica un descuento sobre el monto del curso; el monto final se gestiona externamente.
21. **RN-21.** Una matrícula tiene un estado que refleja el resultado del estudiante (por ejemplo: activa, aprobada, desaprobada).
22. **RN-22.** El docente solo puede marcar asistencia dentro del horario de la clase.

23. **RN-23.** Solo se puede marcar asistencia de estudiantes matriculados en esa sección.
24. **RN-24.** Existe un único registro de asistencia por estudiante y por sesión.
25. **RN-25.** Los estados de asistencia son: presente, ausente, tardanza y justificado.
26. **RN-26.** El material de clase se sube en cada sesión.
27. **RN-27.** Las notas van de 0 a 20; el sistema rechaza cualquier valor fuera de este rango.
28. **RN-28.** Toda sección tiene exactamente tres evaluaciones: Parcial (30%), Continua (40%) y Final (30%).
29. **RN-29.** Existe una única nota por estudiante y por tipo de evaluación.
30. **RN-30.** La nota final se calcula automáticamente solo cuando las tres notas están registradas.
31. **RN-31.** La nota final se calcula como: $(\text{Parcial} \times 0.30) + (\text{Continua} \times 0.40) + (\text{Final} \times 0.30)$.
32. **RN-32.** La nota mínima de aprobación es 11.
33. **RN-33.** Solo el docente asignado a la sección puede registrar o editar las notas de sus estudiantes.
34. **RN-34.** La encuesta de satisfacción docente es opcional para el estudiante.
35. **RN-35.** Pueden responder la encuesta todos los estudiantes matriculados, hayan aprobado o no.
36. **RN-36.** Cada estudiante puede responder la encuesta una sola vez.
37. **RN-37.** Las respuestas de la encuesta son anónimas; no se vinculan con la identidad del estudiante.
38. **RN-38.** El docente es evaluado en varios aspectos (entre 4 y 5) en una escala del 1 al 5.

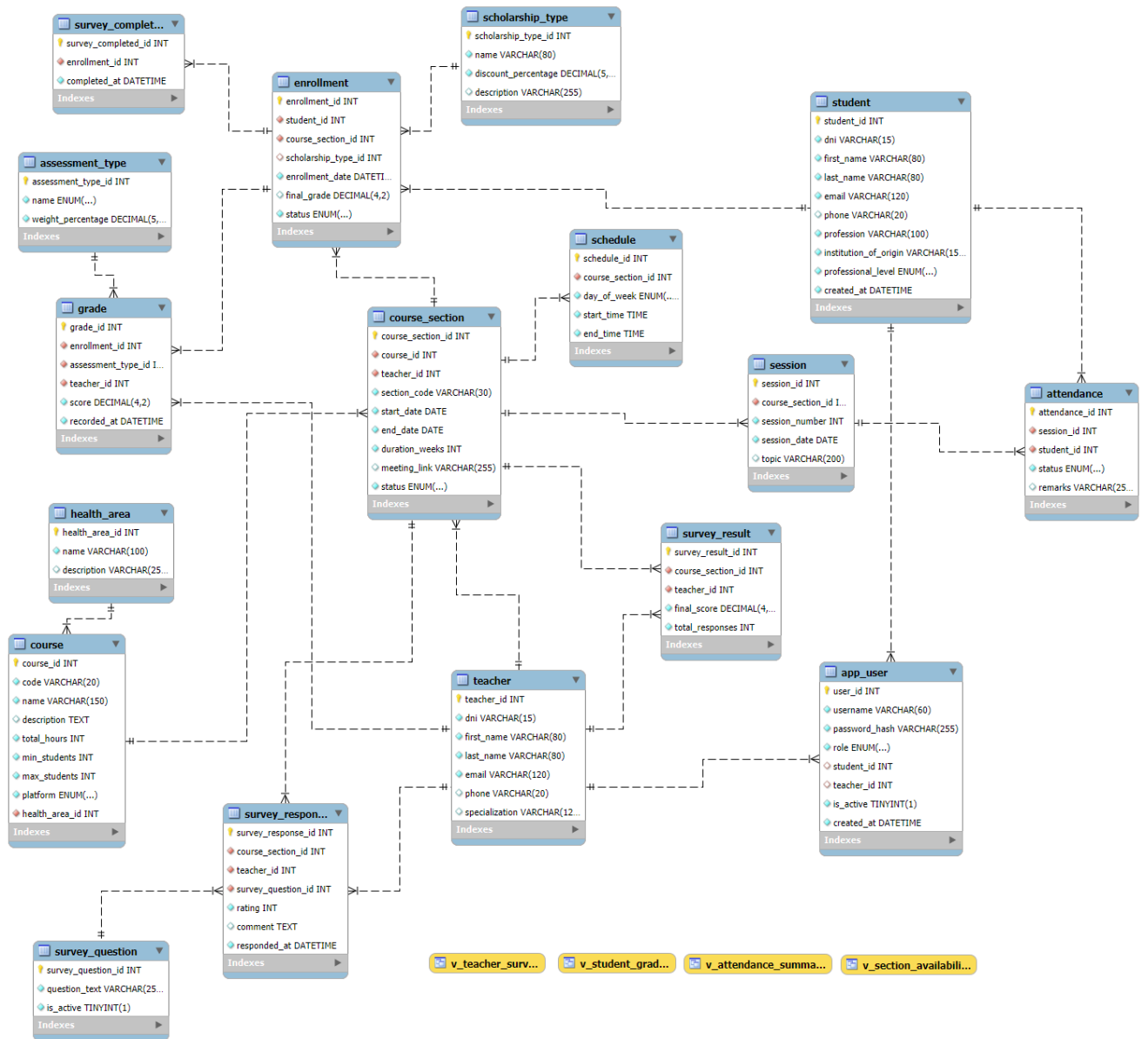
39. **RN-39.** El puntaje final del docente es el promedio de los aspectos evaluados.
40. **RN-40.** El coordinador gestiona el catálogo, las matrículas y los reportes.
41. **RN-41.** El docente gestiona material, asistencia y notas únicamente de sus secciones asignadas.
42. **RN-42.** Todos los objetos de la base de datos se nombran en idioma inglés (política corporativa).

6. Modelos de datos conceptuales





8. Esquema de Base de Datos



8.1 Scripts de generación de esquema Base de Datos

```
-- Create application user
CREATE USER 'cerseu_admin'@'localhost' IDENTIFIED BY 'cerseu2026';

-- Grant privileges over the project database
GRANT ALL PRIVILEGES ON cerseu_med.* TO 'cerseu_admin'@'localhost';

-- Apply changes
FLUSH PRIVILEGES;

-- Create and select the database
DROP DATABASE IF EXISTS cerseu_med;
CREATE DATABASE cerseu_med
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE cerseu_med;
```

8.2 Scripts de generación de objetos de Base de Datos

8.2.1 Creación de tablas

```
-- #TABLE N1: HEALTH_AREA
CREATE TABLE health_area (
    health_area_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255) NULL
) ENGINE=InnoDB;

-- #TABLE N2: TEACHER
CREATE TABLE teacher (
    teacher_id INT AUTO_INCREMENT PRIMARY KEY,
    dni VARCHAR(15) NOT NULL,
    first_name VARCHAR(80) NOT NULL,
    last_name VARCHAR(80) NOT NULL,
    email VARCHAR(120) NOT NULL,
    phone VARCHAR(20) NULL,
    specialization VARCHAR(120) NULL
) ENGINE=InnoDB;

-- #TABLE N3: COURSE
CREATE TABLE course (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    code VARCHAR(20) NOT NULL,
    name VARCHAR(150) NOT NULL,
    description TEXT NULL,
    total_hours INT NOT NULL,
    min_students INT NOT NULL,
    max_students INT NOT NULL,
    platform ENUM('ZOOM', 'GOOGLE_MEET') NOT NULL DEFAULT 'ZOOM',
    health_area_id INT NOT NULL
) ENGINE=InnoDB;

-- #TABLE N4: COURSE_SECTION
CREATE TABLE course_section (
    course_section_id INT AUTO_INCREMENT PRIMARY KEY,
    course_id INT NOT NULL,
    teacher_id INT NOT NULL,
    section_code VARCHAR(30) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    duration_weeks INT NOT NULL,
    meeting_link VARCHAR(255) NULL,
    status ENUM('OPEN', 'IN_PROGRESS', 'CLOSED', 'CANCELLED')
    NOT NULL DEFAULT 'OPEN'
) ENGINE=InnoDB;
```

```

-- #TABLE N5: SCHEDULE
CREATE TABLE schedule (
    schedule_id          INT AUTO_INCREMENT PRIMARY KEY,
    course_section_id    INT NOT NULL,
    day_of_week          ENUM('MONDAY', 'TUESDAY', 'WEDNESDAY', 'THURSDAY',
                              'FRIDAY', 'SATURDAY', 'SUNDAY') NOT NULL,
    start_time           TIME NOT NULL,
    end_time             TIME NOT NULL
) ENGINE=InnoDB;

-- #TABLE N6: STUDENT
CREATE TABLE student (
    student_id           INT AUTO_INCREMENT PRIMARY KEY,
    dni                  VARCHAR(15) NOT NULL,
    first_name           VARCHAR(80) NOT NULL,
    last_name            VARCHAR(80) NOT NULL,
    email                VARCHAR(120) NOT NULL,
    phone                VARCHAR(20) NULL,
    profession            VARCHAR(100) NOT NULL,
    institution_of_origin VARCHAR(150) NOT NULL,
    professional_level    ENUM('STUDENT', 'GRADUATE', 'SPECIALIST') NOT
    NULL,
    created_at           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- #TABLE N7: SCHOLARSHIP_TYPE
CREATE TABLE scholarship_type (
    scholarship_type_id  INT AUTO_INCREMENT PRIMARY KEY,
    name                 VARCHAR(80) NOT NULL,
    discount_percentage  DECIMAL(5,2) NOT NULL,
    description          VARCHAR(255) NULL
) ENGINE=InnoDB;

-- #TABLE N8: ENROLLMENT
CREATE TABLE enrollment (
    enrollment_id        INT AUTO_INCREMENT PRIMARY KEY,
    student_id           INT NOT NULL,
    course_section_id    INT NOT NULL,
    scholarship_type_id  INT NULL,
    enrollment_date       DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    final_grade           DECIMAL(4,2) NULL,
    status                ENUM('ACTIVE', 'APPROVED', 'FAILED') NOT NULL
    DEFAULT 'ACTIVE'
) ENGINE=InnoDB;

-- #TABLE N9: APP_USER
CREATE TABLE app_user (
    user_id              INT AUTO_INCREMENT PRIMARY KEY,
    username              VARCHAR(60) NOT NULL,
    password_hash         VARCHAR(255) NOT NULL,
    role                  ENUM('COORDINATOR', 'TEACHER', 'STUDENT') NOT NULL,
    student_id            INT NULL,
    teacher_id            INT NULL,

```

```

-- #TABLE N10: SESSION
CREATE TABLE session (
    session_id INT AUTO_INCREMENT PRIMARY KEY,
    course_section_id INT NOT NULL,
    session_number INT NOT NULL,
    session_date DATE NOT NULL,
    topic VARCHAR(200) NULL
) ENGINE=InnoDB;

-- #TABLE N11: ATTENDANCE
CREATE TABLE attendance (
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    session_id INT NOT NULL,
    student_id INT NOT NULL,
    status ENUM('PRESENT', 'ABSENT', 'LATE', 'JUSTIFIED') NOT NULL,
    remarks VARCHAR(255) NULL
) ENGINE=InnoDB;

-- #TABLE N12: ASSESSMENT_TYPE
CREATE TABLE assessment_type (
    assessment_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name ENUM('MIDTERM', 'CONTINUOUS', 'FINAL') NOT NULL,
    weight_percentage DECIMAL(5,2) NOT NULL
) ENGINE=InnoDB;

-- #TABLE N13: GRADE
CREATE TABLE grade (
    grade_id INT AUTO_INCREMENT PRIMARY KEY,
    enrollment_id INT NOT NULL,
    assessment_type_id INT NOT NULL,
    teacher_id INT NOT NULL,
    score DECIMAL(4,2) NOT NULL,
    recorded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- #TABLE N14: SURVEY_QUESTION
CREATE TABLE survey_question (
    survey_question_id INT AUTO_INCREMENT PRIMARY KEY,
    question_text VARCHAR(255) NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;

-- #TABLE N15: SURVEY_RESPONSE (anonymous: no student_id stored)
CREATE TABLE survey_response (
    survey_response_id INT AUTO_INCREMENT PRIMARY KEY,
    course_section_id INT NOT NULL,
    teacher_id INT NOT NULL,
    survey_question_id INT NOT NULL,
    rating INT NOT NULL,
    comment TEXT NULL,
    responded_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- #TABLE N16: SURVEY_COMPLETED (participation control, separate from
    answers)
CREATE TABLE survey_completed (
    survey_completed_id INT AUTO_INCREMENT PRIMARY KEY,
    enrollment_id INT NOT NULL,
    completed_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

```

```
-- #TABLE N17: SURVEY_RESULT
CREATE TABLE survey_result (
  survey_result_id INT AUTO_INCREMENT PRIMARY KEY,
  course_section_id INT NOT NULL,
  teacher_id INT NOT NULL,
  final_score DECIMAL(4,2) NOT NULL,
  total_responses INT NOT NULL
) ENGINE=InnoDB;
```

8.2.2 Creación de Llaves Foráneas (FK)

```
-- Add FK to table: COURSE
ALTER TABLE course
  ADD CONSTRAINT fk_course_health_area
  FOREIGN KEY (health_area_id) REFERENCES health_area(health_area_id);

-- Add FK to table: COURSE_SECTION
ALTER TABLE course_section
  ADD CONSTRAINT fk_section_course
  FOREIGN KEY (course_id) REFERENCES course(course_id),
  ADD CONSTRAINT fk_section_teacher
  FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id);

-- Add FK to table: SCHEDULE
ALTER TABLE schedule
  ADD CONSTRAINT fk_schedule_section
  FOREIGN KEY (course_section_id) REFERENCES course_section(
    course_section_id) ON DELETE CASCADE;

-- Add FK to table: ENROLLMENT
ALTER TABLE enrollment
  ADD CONSTRAINT fk_enrollment_student
  FOREIGN KEY (student_id) REFERENCES student(student_id),
  ADD CONSTRAINT fk_enrollment_section
  FOREIGN KEY (course_section_id) REFERENCES course_section(
    course_section_id),
  ADD CONSTRAINT fk_enrollment_scholarship
  FOREIGN KEY (scholarship_type_id) REFERENCES scholarship_type(
    scholarship_type_id);

-- Add FK to table: APP_USER
ALTER TABLE app_user
  ADD CONSTRAINT fk_user_student
  FOREIGN KEY (student_id) REFERENCES student(student_id),
  ADD CONSTRAINT fk_user_teacher
  FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id);

-- Add FK to table: SESSION
ALTER TABLE session
  ADD CONSTRAINT fk_session_section
  FOREIGN KEY (course_section_id) REFERENCES course_section(
    course_section_id) ON DELETE CASCADE;

-- Add FK to table: ATTENDANCE
ALTER TABLE attendance
  ADD CONSTRAINT fk_attendance_session
  FOREIGN KEY (session_id) REFERENCES session(session_id) ON DELETE
  CASCADE,
  ADD CONSTRAINT fk_attendance_student
  FOREIGN KEY (student_id) REFERENCES student(student_id);
```



```

-- Add FK to table: GRADE
ALTER TABLE grade
    ADD CONSTRAINT fk_grade_enrollment
        FOREIGN KEY (enrollment_id) REFERENCES enrollment(enrollment_id)
        ON DELETE CASCADE,
    ADD CONSTRAINT fk_grade_assessment
        FOREIGN KEY (assessment_type_id) REFERENCES assessment_type(
            assessment_type_id),
    ADD CONSTRAINT fk_grade_teacher
        FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id);

-- Add FK to table: SURVEY_RESPONSE
ALTER TABLE survey_response
    ADD CONSTRAINT fk_response_section
        FOREIGN KEY (course_section_id) REFERENCES course_section(
            course_section_id),
    ADD CONSTRAINT fk_response_teacher
        FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id),
    ADD CONSTRAINT fk_response_question
        FOREIGN KEY (survey_question_id) REFERENCES survey_question(
            survey_question_id);

-- Add FK to table: SURVEY_COMPLETED
ALTER TABLE survey_completed
    ADD CONSTRAINT fk_completed_enrollment
        FOREIGN KEY (enrollment_id) REFERENCES enrollment(enrollment_id);

-- Add FK to table: SURVEY_RESULT
ALTER TABLE survey_result
    ADD CONSTRAINT fk_result_section
        FOREIGN KEY (course_section_id) REFERENCES course_section(
            course_section_id),
    ADD CONSTRAINT fk_result_teacher
        FOREIGN KEY (teacher_id) REFERENCES teacher(teacher_id);

```

8.2.3 Creación de Restricciones (UNIQUE/CHECK)

```

-- Add UNIQUE constraints
ALTER TABLE teacher
    ADD CONSTRAINT uq_teacher_dni    UNIQUE (dni),
    ADD CONSTRAINT uq_teacher_email UNIQUE (email);

ALTER TABLE course
    ADD CONSTRAINT uq_course_code UNIQUE (code);

ALTER TABLE course_section
    ADD CONSTRAINT uq_section_code UNIQUE (section_code);

ALTER TABLE student
    ADD CONSTRAINT uq_student_dni    UNIQUE (dni),
    ADD CONSTRAINT uq_student_email UNIQUE (email);

ALTER TABLE enrollment
    ADD CONSTRAINT uq_enrollment UNIQUE (student_id, course_section_id);

ALTER TABLE app_user
    ADD CONSTRAINT uq_user_username UNIQUE (username);

ALTER TABLE session
    ADD CONSTRAINT uq_session UNIQUE (course_section_id, session_number);

ALTER TABLE attendance
    ADD CONSTRAINT uq_attendance UNIQUE (session_id, student_id);

```



```

ALTER TABLE assessment_type
    ADD CONSTRAINT uq_assessment_name UNIQUE (name);

ALTER TABLE grade
    ADD CONSTRAINT uq_grade UNIQUE (enrollment_id, assessment_type_id);

ALTER TABLE survey_completed
    ADD CONSTRAINT uq_completed_enrollment UNIQUE (enrollment_id);

ALTER TABLE survey_result
    ADD CONSTRAINT uq_result_section UNIQUE (course_section_id);

```

```

-- Add CHECK constraints
ALTER TABLE course
    ADD CONSTRAINT chk_course_capacity CHECK (max_students >=
        min_students),
    ADD CONSTRAINT chk_course_hours CHECK (total_hours > 0);

ALTER TABLE course_section
    ADD CONSTRAINT chk_section_dates CHECK (end_date > start_date);

ALTER TABLE schedule
    ADD CONSTRAINT chk_schedule_time CHECK (end_time > start_time);

ALTER TABLE scholarship_type
    ADD CONSTRAINT chk_scholarship_pct CHECK (discount_percentage BETWEEN
        0 AND 100);

ALTER TABLE enrollment
    ADD CONSTRAINT chk_enrollment_final_grade
    CHECK (final_grade IS NULL OR (final_grade BETWEEN 0 AND 20));

ALTER TABLE assessment_type
    ADD CONSTRAINT chk_assessment_weight CHECK (weight_percentage BETWEEN
        0 AND 100);

ALTER TABLE grade
    ADD CONSTRAINT chk_grade_score CHECK (score BETWEEN 0 AND 20);

ALTER TABLE survey_response
    ADD CONSTRAINT chk_response_rating CHECK (rating BETWEEN 1 AND 5);

```

8.2.3 Creación de índices

```

-- Performance indexes
CREATE INDEX idx_section_course ON course_section(course_id);
CREATE INDEX idx_section_teacher ON course_section(teacher_id);
CREATE INDEX idx_enrollment_sec ON enrollment(course_section_id);
CREATE INDEX idx_enrollment_stu ON enrollment(student_id);
CREATE INDEX idx_session_section ON session(course_section_id);
CREATE INDEX idx_grade_enrollment ON grade(enrollment_id);
CREATE INDEX idx_response_section ON survey_response(course_section_id);

```

8.2.4 Creación de Funciones

```
DELIMITER $$

-- #FUNCTION N1: weighted final grade of an enrollment (RN-31, RN-32)
DROP FUNCTION IF EXISTS fn_final_grade $$
CREATE FUNCTION fn_final_grade(p_enrollment_id INT)
RETURNS DECIMAL(4,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_count          INT;
    DECLARE v_final_grade DECIMAL(6,2);

    SELECT COUNT(*) INTO v_count
    FROM grade
    WHERE enrollment_id = p_enrollment_id;

    IF v_count < 3 THEN
        RETURN NULL;
    END IF;

    SELECT SUM(g.score * at.weight_percentage / 100) INTO v_final_grade
    FROM grade g
    JOIN assessment_type at ON g.assessment_type_id = at.
        assessment_type_id
    WHERE g.enrollment_id = p_enrollment_id;

    RETURN ROUND(v_final_grade, 2);
END $$

-- #FUNCTION N2: attendance percentage of an enrollment (PRESENT + LATE)
DROP FUNCTION IF EXISTS fn_attendance_rate $$
CREATE FUNCTION fn_attendance_rate(p_enrollment_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_student_id INT;
    DECLARE v_section_id INT;
    DECLARE v_total      INT;
    DECLARE v_present    INT;

    SELECT student_id, course_section_id
        INTO v_student_id, v_section_id
    FROM enrollment
    WHERE enrollment_id = p_enrollment_id;

    SELECT COUNT(*) INTO v_total
    FROM session
    WHERE course_section_id = v_section_id;

    IF v_total = 0 THEN
        RETURN 0;
    END IF;
```

```

SELECT COUNT(*) INTO v_present
FROM attendance a
JOIN session s ON a.session_id = s.session_id
WHERE s.course_section_id = v_section_id
      AND a.student_id = v_student_id
      AND a.status IN ('PRESENT', 'LATE');

RETURN ROUND(v_present * 100 / v_total, 2);
END $$

-- #FUNCTION N3: detect schedule conflict before enrolling (RN-16)
DROP FUNCTION IF EXISTS fn_has_schedule_conflict $$
CREATE FUNCTION fn_has_schedule_conflict(p_student_id INT, p_section_id
INT)
RETURNS TINYINT
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_conflicts INT;

    SELECT COUNT(*) INTO v_conflicts
    FROM enrollment e
    JOIN course_section cs ON e.course_section_id = cs.course_section_id
    JOIN schedule s ON cs.course_section_id = s.course_section_id
    JOIN schedule ns ON ns.course_section_id = p_section_id
    JOIN course_section ncs ON ncs.course_section_id = p_section_id
    WHERE e.student_id = p_student_id
          AND e.status = 'ACTIVE'
          AND e.course_section_id <> p_section_id
          AND s.day_of_week = ns.day_of_week
          AND s.start_time < ns.end_time
          AND ns.start_time < s.end_time
          AND cs.start_date <= ncs.end_date
          AND ncs.start_date <= cs.end_date;

    RETURN IF(v_conflicts > 0, 1, 0);
END $$

```

8.2.5 Creación de Procedimientos

```

-- #PROCEDURE N1: register a new student and create the STUDENT account
DROP PROCEDURE IF EXISTS sp_register_student $$
CREATE PROCEDURE sp_register_student(
    IN  p_dni          VARCHAR(15),
    IN  p_first_name   VARCHAR(80),
    IN  p_last_name    VARCHAR(80),
    IN  p_email        VARCHAR(120),
    IN  p_phone        VARCHAR(20),
    IN  p_profession    VARCHAR(100),
    IN  p_institution   VARCHAR(150),
    IN  p_level        VARCHAR(20),
    IN  p_username     VARCHAR(60),
    IN  p_password_hash VARCHAR(255),
    OUT p_student_id   INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

```

```

START TRANSACTION;

INSERT INTO student(dni, first_name, last_name, email, phone,
                    profession, institution_of_origin,
                    professional_level)
VALUES (p_dni, p_first_name, p_last_name, p_email, p_phone,
        p_profession, p_institution, p_level);

SET p_student_id = LAST_INSERT_ID();

INSERT INTO app_user(username, password_hash, role, student_id)
VALUES (p_username, p_password_hash, 'STUDENT', p_student_id);

COMMIT;
END $$

-- #PROCEDURE N2: enroll a student validating RN-08, RN-16, RN-17, RN-18
DROP PROCEDURE IF EXISTS sp_enroll_student $$
CREATE PROCEDURE sp_enroll_student(
    IN p_student_id      INT,
    IN p_section_id      INT,
    IN p_scholarship_type_id INT,
    OUT p_enrollment_id  INT
)
BEGIN
    DECLARE v_status      VARCHAR(15);
    DECLARE v_max         INT;
    DECLARE v_current     INT;
    DECLARE v_duplicate   INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- RN-08: section must be OPEN
    SELECT cs.status, c.max_students
        INTO v_status, v_max
    FROM course_section cs
    JOIN course c ON cs.course_id = c.course_id
    WHERE cs.course_section_id = p_section_id
    FOR UPDATE;

    IF v_status IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Section does not
            exist.';
    END IF;
    IF v_status <> 'OPEN' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Section is not open
            for enrollment.';
    END IF;

```

```

END IF;

-- RN-17: no duplicate enrollment
SELECT COUNT(*) INTO v_duplicate
FROM enrollment
WHERE student_id = p_student_id AND course_section_id = p_section_id;
IF v_duplicate > 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student is already
        enrolled in this section.';
END IF;

-- RN-18: capacity not exceeded
SELECT COUNT(*) INTO v_current
FROM enrollment
WHERE course_section_id = p_section_id AND status = 'ACTIVE';
IF v_current >= v_max THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Section has reached
        its maximum capacity.';
END IF;

-- RN-16: no schedule conflict
IF fn_has_schedule_conflict(p_student_id, p_section_id) = 1 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Schedule conflict
        with another active enrollment.';
END IF;

INSERT INTO enrollment(student_id, course_section_id,
    scholarship_type_id)
VALUES (p_student_id, p_section_id, p_scholarship_type_id);

SET p_enrollment_id = LAST_INSERT_ID();

COMMIT;
END $$

-- #PROCEDURE N3: register a grade (RN-28, RN-34) and recompute final
    grade
DROP PROCEDURE IF EXISTS sp_register_grade $$
CREATE PROCEDURE sp_register_grade(
    IN p_enrollment_id    INT,
    IN p_assessment_type_id INT,
    IN p_teacher_id       INT,
    IN p_score             DECIMAL(4,2)
)
BEGIN
    DECLARE v_section_teacher INT;
    DECLARE v_final           DECIMAL(4,2);

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    -- RN-28: score range
    IF p_score < 0 OR p_score > 20 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Score must be between
            0 and 20.';
    END IF;

```

```

-- RN-34: only the assigned teacher can register grades
SELECT cs.teacher_id INTO v_section_teacher
FROM enrollment e
JOIN course_section cs ON e.course_section_id = cs.course_section_id
WHERE e.enrollment_id = p_enrollment_id;

IF v_section_teacher IS NULL THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Enrollment does not
    exist.';
END IF;
IF v_section_teacher <> p_teacher_id THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Teacher is not
    assigned to this section.';
END IF;

-- RN-30: one grade per assessment type (insert or update)
INSERT INTO grade(enrollment_id, assessment_type_id, teacher_id,
    score)
VALUES (p_enrollment_id, p_assessment_type_id, p_teacher_id, p_score)
ON DUPLICATE KEY UPDATE score = p_score, recorded_at =
    CURRENT_TIMESTAMP;

-- RN-31/32/33: recompute final grade when the three grades exist
SET v_final = fn_final_grade(p_enrollment_id);
IF v_final IS NOT NULL THEN
    UPDATE enrollment
    SET final_grade = v_final,
        status = IF(v_final >= 11, 'APPROVED', 'FAILED')
    WHERE enrollment_id = p_enrollment_id;
END IF;

COMMIT;
END $$

```

```

-- #PROCEDURE N4: mark attendance validating enrollment (RN-24, RN-25)
DROP PROCEDURE IF EXISTS sp_mark_attendance $$
CREATE PROCEDURE sp_mark_attendance(
    IN p_session_id INT,
    IN p_student_id INT,
    IN p_status      VARCHAR(15),
    IN p_remarks     VARCHAR(255)
)
BEGIN
    DECLARE v_enrolled INT;

    SELECT COUNT(*) INTO v_enrolled
    FROM session s
    JOIN enrollment e ON e.course_section_id = s.course_section_id
    WHERE s.session_id = p_session_id
        AND e.student_id = p_student_id;

    IF v_enrolled = 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Student is not
        enrolled in this section.';
    END IF;

    INSERT INTO attendance(session_id, student_id, status, remarks)
    VALUES (p_session_id, p_student_id, p_status, p_remarks)
    ON DUPLICATE KEY UPDATE status = p_status, remarks = p_remarks;
END $$

```



```

-- #PROCEDURE N5: submit an anonymous survey, single submission (RN-42,
RN-43)
DROP PROCEDURE IF EXISTS sp_submit_survey $$
CREATE PROCEDURE sp_submit_survey(
    IN p_enrollment_id INT,
    IN p_q1 INT, IN p_q2 INT, IN p_q3 INT, IN p_q4 INT, IN p_q5 INT,
    IN p_comment TEXT
)
BEGIN
    DECLARE v_section_id INT;
    DECLARE v_teacher_id INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT cs.course_section_id, cs.teacher_id
        INTO v_section_id, v_teacher_id
    FROM enrollment e
    JOIN course_section cs ON e.course_section_id = cs.course_section_id
    WHERE e.enrollment_id = p_enrollment_id;

    IF v_section_id IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Enrollment does not
        exist.';
    END IF;

    -- RN-42: single submission enforced by UNIQUE(enrollment_id)
    INSERT INTO survey_completed(enrollment_id) VALUES (p_enrollment_id);

    -- RN-43: anonymous responses (no student_id stored)
    INSERT INTO survey_response(course_section_id, teacher_id,
        survey_question_id, rating, comment)
    VALUES
        (v_section_id, v_teacher_id, 1, p_q1, NULL),
        (v_section_id, v_teacher_id, 2, p_q2, NULL),
        (v_section_id, v_teacher_id, 3, p_q3, NULL),
        (v_section_id, v_teacher_id, 4, p_q4, NULL),
        (v_section_id, v_teacher_id, 5, p_q5, p_comment);

    COMMIT;
END $$

```

```

-- #PROCEDURE N6: consolidate teacher survey result (RN-45)
DROP PROCEDURE IF EXISTS sp_calculate_survey_result $$
CREATE PROCEDURE sp_calculate_survey_result(IN p_section_id INT)
BEGIN
    DECLARE v_teacher_id INT;
    DECLARE v_avg          DECIMAL(4,2);
    DECLARE v_total        INT;

    SELECT teacher_id INTO v_teacher_id
    FROM course_section WHERE course_section_id = p_section_id;

    SELECT ROUND(AVG(rating),2) INTO v_avg
    FROM survey_response
    WHERE course_section_id = p_section_id;

```

```

SELECT COUNT(*) INTO v_total
FROM survey_completed sc
JOIN enrollment e ON sc.enrollment_id = e.enrollment_id
WHERE e.course_section_id = p_section_id;

INSERT INTO survey_result(course_section_id, teacher_id, final_score,
    total_responses)
VALUES (p_section_id, v_teacher_id, IFNULL(v_avg,0), v_total)
ON DUPLICATE KEY UPDATE
    final_score = IFNULL(v_avg,0),
    total_responses = v_total;
END $$

```

8.2.6 Creación de Vistas

```

DELIMITER ;

-- #VIEW N1: open sections with current occupancy
CREATE OR REPLACE VIEW v_section_availability AS
SELECT cs.course_section_id,
    c.name AS course_name,
    cs.section_code,
    CONCAT(t.first_name, ' ', t.last_name) AS teacher,
    c.max_students,
    COUNT(e.enrollment_id) AS enrolled,
    c.max_students - COUNT(e.enrollment_id) AS available_seats,
    cs.status
FROM course_section cs
JOIN course c ON cs.course_id = c.course_id
JOIN teacher t ON cs.teacher_id = t.teacher_id
LEFT JOIN enrollment e
    ON e.course_section_id = cs.course_section_id AND e.status = '
    ACTIVE'
GROUP BY cs.course_section_id, c.name, cs.section_code,
    teacher, c.max_students, cs.status;

-- #VIEW N2: student grades consolidated by section
CREATE OR REPLACE VIEW v_student_grades AS
SELECT e.enrollment_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student,
    c.name AS course_name,
    cs.section_code,
    MAX(CASE WHEN at.name='MIDTERM' THEN g.score END) AS midterm,
    MAX(CASE WHEN at.name='CONTINUOUS' THEN g.score END) AS
        continuous_eval,
    MAX(CASE WHEN at.name='FINAL' THEN g.score END) AS final_exam
    ,
    e.final_grade,
    e.status
FROM enrollment e
JOIN student s ON e.student_id = s.student_id
JOIN course_section cs ON e.course_section_id = cs.course_section_id
JOIN course c ON cs.course_id = c.course_id
LEFT JOIN grade g ON g.enrollment_id = e.enrollment_id
LEFT JOIN assessment_type at ON g.assessment_type_id = at.
    assessment_type_id
GROUP BY e.enrollment_id, student, c.name, cs.section_code,
    e.final_grade, e.status;

```



```

-- #VIEW N3: attendance summary per enrollment
CREATE OR REPLACE VIEW v_attendance_summary AS
SELECT e.enrollment_id,
       CONCAT(s.first_name, ' ', s.last_name) AS student,
       c.name AS course_name,
       cs.section_code,
       fn_attendance_rate(e.enrollment_id) AS attendance_rate
FROM enrollment e
JOIN student s      ON e.student_id = s.student_id
JOIN course_section cs ON e.course_section_id = cs.course_section_id
JOIN course c       ON cs.course_id = c.course_id;

-- #VIEW N5: teacher satisfaction report
CREATE OR REPLACE VIEW v_teacher_survey AS
SELECT CONCAT(t.first_name, ' ', t.last_name) AS teacher,
       c.name AS course_name,
       cs.section_code,
       sr.final_score,
       sr.total_responses
FROM survey_result sr
JOIN course_section cs ON sr.course_section_id = cs.course_section_id
JOIN course c         ON cs.course_id = c.course_id
JOIN teacher t        ON sr.teacher_id = t.teacher_id;

```

8.3 Scripts de carga de Datos

```

-- HEALTH AREAS
INSERT INTO health_area(name, description) VALUES
('Public Health',      'Epidemiology and community health'),
('Clinical Skills',    'Clinical and assistance skills'),
('Research',           'Scientific research in health'),
('Mental Health',     'Mental health and psychological well-being'),
('Nutrition',          'Clinical nutrition and dietetics'),
('Health Management', 'Health administration and management'),
('Medical Technology', 'Medical technology and informatics');

-- ASSESSMENT TYPES (fixed weights: RN-29)
INSERT INTO assessment_type(name, weight_percentage) VALUES
('MIDTERM',    30.00),
('CONTINUOUS', 40.00),
('FINAL',      30.00);

-- SCHOLARSHIP TYPES
INSERT INTO scholarship_type(name, discount_percentage, description)
VALUES
('Full Scholarship',    100.00, 'Full coverage for vulnerable cases'),
('Partial Scholarship',  50.00, 'Half coverage'),
('Social Scholarship',   75.00, 'High social vulnerability case'),
('Excellence',          30.00, 'Top performers from previous editions');

```

```

-- SURVEY QUESTIONS
INSERT INTO survey_question(question_text, is_active) VALUES
('The teacher masters the course topics', TRUE),
('The teacher is punctual and follows schedule', TRUE),
('The teacher explains clearly', TRUE),
('The materials and resources were useful', TRUE),
('The teacher answers questions appropriately', TRUE);

-- TEACHERS
INSERT INTO teacher(dni, first_name, last_name, email, phone,
specialization) VALUES
('41258963', 'Luis', 'Baldeon', 'lbaldeon@cerseu.pe', '987654321', 'Public
Health'),
('40789654', 'Jorge', 'Solano', 'jsolano@cerseu.pe', '986554120', 'Medical
Technology'),
('42365871', 'Maria', 'Quispe', 'mquispe@cerseu.pe', '985236471', 'Mental
Health'),
('43654128', 'Carlos', 'Rojas', 'crojas@cerseu.pe', '984125874', 'Research');

-- TEACHER ACCOUNTS
INSERT INTO app_user(username, password_hash, role, teacher_id) VALUES
('lbaldeon', '$2y$10$hashLB', 'TEACHER', 1),
('jsolano', '$2y$10$hashJS', 'TEACHER', 2),
('mquispe', '$2y$10$hashMQ', 'TEACHER', 3),
('crojas', '$2y$10$hashCR', 'TEACHER', 4);

-- COORDINATOR ACCOUNT
INSERT INTO app_user(username, password_hash, role) VALUES
('coordinator', '$2y$10$hashCO', 'COORDINATOR');

-- COURSES
INSERT INTO course(code, name, description, total_hours, min_students,
max_students, platform, health_area_id) VALUES
('CRS-EPI', 'Applied Epidemiology', 'Fundamentals of epidemiology
applied to public health.', 48, 5, 30, 'ZOOM', 1),
('CRS-BST', 'Biostatistics for Health', 'Statistics applied to health
research.', 40, 5, 25, 'GOOGLE_MEET', 3),
('CRS-MH', 'Mental Health First Aid', 'Basic psychological support
skills.', 32, 4, 20, 'ZOOM', 4);

-- COURSE SECTIONS (Section A: open / MH-2024B: closed for cert & survey
tests)
INSERT INTO course_section(course_id, teacher_id, section_code,
start_date, end_date, duration_weeks, meeting_link, status) VALUES
(1, 1, 'EPI-2025A', '2025-03-03', '2025-04-25', 8, 'https://zoom.us/j/
epi2025a', 'OPEN'),
(2, 4, 'BST-2025A', '2025-03-04', '2025-05-27', 12, 'https://meet.google.
com/bst', 'OPEN'),
(3, 3, 'MH-2024B', '2024-09-02', '2024-10-25', 8, 'https://zoom.us/j/
mh2024b', 'CLOSED');

-- SCHEDULES
-- EPI-2025A: Mon & Wed 14:00-17:00
INSERT INTO schedule(course_section_id, day_of_week, start_time, end_time
) VALUES
(1, 'MONDAY', '14:00:00', '17:00:00'),
(1, 'WEDNESDAY', '14:00:00', '17:00:00');

```

```

-- BST-2025A: Tue & Thu 14:00-16:00
INSERT INTO schedule(course_section_id, day_of_week, start_time, end_time
) VALUES
(2, 'TUESDAY', '14:00:00', '16:00:00'),
(2, 'THURSDAY', '14:00:00', '16:00:00');
-- MH-2024B: Mon & Wed 18:00-21:00
INSERT INTO schedule(course_section_id, day_of_week, start_time, end_time
) VALUES
(3, 'MONDAY', '18:00:00', '21:00:00'),
(3, 'WEDNESDAY', '18:00:00', '21:00:00');

-- STUDENTS
INSERT INTO student(dni, first_name, last_name, email, phone, profession,
institution_of_origin, professional_level) VALUES
('70123456', 'Ana', 'Torres', 'atorres@mail.com', '911111111', 'Nurse', 'UNMSM',
'GRADUATE'),
('70234567', 'Pedro', 'Gomez', 'pgomez@mail.com', '922222222', 'Medical
Student', 'UPCH', 'STUDENT'),
('70345678', 'Lucia', 'Mendoza', 'lmendoza@mail.com', '933333333', '
Obstetrician', 'UNFV', 'SPECIALIST'),
('70456789', 'Diego', 'Ramos', 'dramos@mail.com', '944444444', 'Psychologist',
'UNMSM', 'GRADUATE'),
('70567890', 'Sofia', 'Vega', 'svega@mail.com', '955555555', 'Nutritionist', '
USMP', 'GRADUATE');

-- STUDENT ACCOUNTS
INSERT INTO app_user(username, password_hash, role, student_id) VALUES
('atorres', '$2y$10$hashAT', 'STUDENT', 1),
('pgomez', '$2y$10$hashPG', 'STUDENT', 2),
('lmendoza', '$2y$10$hashLM', 'STUDENT', 3),
('dramos', '$2y$10$hashDR', 'STUDENT', 4),
('svega', '$2y$10$hashSV', 'STUDENT', 5);

-- ENROLLMENTS for the CLOSED section MH-2024B
INSERT INTO enrollment(student_id, course_section_id, scholarship_type_id
) VALUES
(4, 3, NULL), -- Diego -> MH-2024B (enrollment_id = 5)
(5, 3, 3); -- Sofia -> MH-2024B (enrollment_id = 6, social
scholarship)

-- SESSIONS (only session dates; time comes from schedule)
INSERT INTO session(course_section_id, session_number, session_date,
topic) VALUES
(1, 1, '2025-03-03', 'Introduction to Epidemiology'),
(1, 2, '2025-03-05', 'Measures of Frequency'),
(3, 1, '2024-09-02', 'Principles of Mental Health'),
(3, 2, '2024-09-04', 'Crisis Identification');

-- ATTENDANCE (closed section, for reporting)
INSERT INTO attendance(session_id, student_id, status) VALUES
(3, 4, 'PRESENT'),
(3, 5, 'PRESENT'),
(4, 4, 'LATE'),
(4, 5, 'ABSENT');

```

```

-- GRADES for the CLOSED section (via stored procedure to compute final
  grade)
-- assessment_type: 1=MIDTERM 2=CONTINUOUS 3=FINAL ; teacher 3 owns
  section 3
CALL sp_register_grade(5, 1, 3, 16); -- Diego midterm
CALL sp_register_grade(5, 2, 3, 15); -- Diego continuous
CALL sp_register_grade(5, 3, 3, 14); -- Diego final      -> final 15.00
  APPROVED

CALL sp_register_grade(6, 1, 3, 9);   -- Sofia midterm
CALL sp_register_grade(6, 2, 3, 10);  -- Sofia continuous
CALL sp_register_grade(6, 3, 3, 8);   -- Sofia final      -> final 9.10
  FAILED

-- SURVEYS for the CLOSED section (anonymous) + result consolidation
CALL sp_submit_survey(5, 5,5,4,5,4,'Excellent course');
CALL sp_submit_survey(6, 4,5,4,4,5,'Very useful');
CALL sp_calculate_survey_result(3);

```